

# **DEFENCE RESEARCH AND DEVELOPMENT ORGANISATION**



## **AN INTERNSHIP PROJECT REPORT ON NUMERICAL SIMULATION OF C-D NOZZLE USING CFD AND NUMERICAL SOLUTION OF TRANSIENT RADIAL HEAT CONDUCTION**

*Submitted by*

**V. VARSHITHA PREETHAM**

**22ME01028**

IN THE PARTIAL FULFILMENT OF THE DEGREE OF  
**‘BACHELOR OF TECHNOLOGY’**  
**(MECHANICAL ENGINEERING)**  
OF  
**INDIAN INSTITUTE OF TECHNOLOGY BHUBANESWAR**

## **DEFENCE RESEARCH AND DEVELOPMENT ORGANISATION (DRDO)**

DRDO is the R&D wing of Ministry of Defence, Govt of India, with a vision to empower India with cutting-edge defence technologies and a mission to achieve self-reliance in critical defence technologies and systems, while equipping their armed forces with state-of-the-art weapon systems and equipment in accordance with requirements laid down by the three Services. DRDO's pursuit of self-reliance and successful indigenous development and production of strategic systems and platforms such as Agni and Prithvi series of missiles; light combat aircraft, Tejas; multi-barrel rocket launcher, Pinaka; air defence system, Akash; a wide range of radars and electronic warfare systems; etc., have given quantum jump to India's military might, generating effective deterrence and providing crucial leverage.

"Balasya Mulam Vigyanam"—the source of strength is science—drives the nation in peace and war. DRDO has firm determination to make the nation strong and self-reliant in terms of science and technology, especially in the field of military technologies.

DRDO was formed in 1958 from the amalgamation of the then already functioning Technical Development Establishment (TDEs) of the Indian Army and the Directorate of Technical Development & Production (DTDP) with the Defence Science Organisation (DSO). DRDO was then a small organisation with 10 establishments or laboratories. Over the years, it has grown multi-directionally in terms of the variety of subject disciplines, number of laboratories, achievements and stature.

Today, DRDO is a network of around 41 laboratories and 05 DRDO Young Scientist Laboratories (DYSLs) which are deeply engaged in developing defence technologies covering various disciplines, like aeronautics, armaments, electronics, combat vehicles, engineering systems, instrumentation, missiles, advanced computing and simulation, special materials, naval systems, life sciences, training, information systems and agriculture. Several major projects for the development of missiles, armaments, light combat aircrafts, radars, electronic warfare systems etc are on hand and significant achievements have already been made in several such technologies.

Place : DRDO ASL (Advanced Systems Laboratory), Hyderabad

Date : 26-07-2025

## **Topic I : Numerical Simulation of C-D nozzle using CFD (Computational Fluid Dynamics) (CFD++ v/s Empirical) – Development of Python code for solving Mach number**

### **Abstract:**

The numerical investigation of a rocket-class convergent–divergent (C-D) nozzle was conducted through a two-pronged approach, combining high-fidelity CFD++ simulations with an in-house Python-based empirical solver. The CFD++ analysis was performed under axisymmetric, viscous, steady-state compressible flow conditions representative of realistic operating scenarios, with an inlet temperature of 2800 K, inlet pressure of 40 bar, inlet velocity of 10 m/s, and isothermal nozzle walls maintained at 300 K. The simulation generated spatially resolved distributions of Mach number, pressure, and temperature throughout the nozzle, clearly capturing the acceleration of subsonic flow in the convergent section, sonic flow at the throat, and supersonic expansion in the divergent section. The CFD++ solution predicted a maximum exit Mach number of 3.7296, providing a reliable reference for validating analytical and semi-empirical approaches.

In parallel, a Python program was developed to compute the Mach number distribution along the nozzle using the quasi–one–dimensional area–Mach number relationship. The equation was solved iteratively via the bisection method, which ensured convergence for both subsonic and supersonic regions. The code was designed with modularity, allowing user-defined inputs for the specific heat ratio, gas constant, and throat conditions, thereby enabling rapid sensitivity analysis and design optimisation. Applied to the same nozzle geometry as the CFD++ model, the Python solver predicted an exit Mach number of 3.5676, corresponding to a deviation of only 4.5% from the CFD++ result. This close agreement demonstrates the robustness and reliability of the empirical approach, establishing it as an effective tool for preliminary nozzle design, performance evaluation, and cross-validation of CFD outputs.

### **Keywords:**

- Convergent–Divergent Nozzle
- Bisection Method
- Area–Mach Number Relationship
- Compressible Flow

## **Topic II : Numerical Solution of Transient Radial Heat Conduction using Finite Volume Method (FVM) – Ansys APDL v/s Python code**

### **Abstract:**

The transient radial heat conduction in the annular wall section of a nozzle was analysed using the finite volume method (FVM) and validated against results obtained from Ansys APDL simulations. The study considered an inner radius of 2.005 m and an outer radius of 2.010 m, with constant material properties—density of 1400 kg/m<sup>3</sup>, specific heat of 1000 J/kg·K, and thermal conductivity of 0.1 W/m·K. The boundary conditions specified a constant heat flux of 20,000 W/m<sup>2</sup> applied at the inner wall and zero flux at the outer wall. A uniform initial temperature of 300 K was assumed. The Python code, developed for this purpose, discretised the unsteady one-dimensional radial heat conduction equation in cylindrical coordinates using an implicit scheme to ensure stability. The Tri-Diagonal Matrix Algorithm (TDMA) was employed for solving the resulting algebraic equations efficiently, enabling calculation of the temperature distribution over a simulation period of 30 seconds.

The simulation results from the Python code were compared with those obtained from Ansys APDL, where a 2D annular section with a small included angle (0.1 rad) was meshed using Quad 4-node 55 thermal elements. Identical physical parameters and boundary conditions were applied in both methods. The comparison revealed excellent agreement, with both approaches predicting a maximum temperature of approximately 630 K at the inner wall and a minimum temperature of about 304 K at the outer wall after 30 seconds. The close correspondence between the Python and Ansys APDL results confirms the accuracy and reliability of the developed computational model. This validated Python implementation provides a flexible and efficient tool for transient heat conduction analysis in cylindrical geometries, enabling rapid parametric studies and supporting thermal design optimisation in engineering applications.

### **Keywords:**

- Transient Heat Conduction
- Finite Volume Method
- Tri-Diagonal Matrix Algorithm (TDMA)
- Implicit Scheme

## CONTENTS

### TOPIC I :

| Contents                                                | Page No. |
|---------------------------------------------------------|----------|
| INTRODUCTION                                            |          |
| 1.1 Convergent-Divergent Nozzle                         |          |
| PROBLEM STATEMENT                                       |          |
| METHODOLOGY                                             |          |
| 3.1 Python code for Area-Mach relationship              |          |
| 3.1.1 Bisection method                                  |          |
| 3.1.2 Nozzle Geometry                                   |          |
| 3.1.3 Representing Area-Mach relationship as a function |          |
| 3.1.4 Python code                                       |          |
| 3.2 CFD++ simulation                                    |          |
| 3.2.1 Setup synopsis                                    |          |
| RESULTS                                                 |          |
| 4.1 Meshing                                             |          |
| 4.2 CFD++                                               |          |
| 4.3 Python code                                         |          |
| C-D NOZZLE IN ROCKET PROPULSION                         |          |
| CONCLUSION/ SUMMARY                                     |          |

**TOPIC II :**

| <b>Contents</b>                                      | <b>Page No.</b> |
|------------------------------------------------------|-----------------|
| INTRODUCTION                                         |                 |
| 1.1 Modes of Heat Transfer                           |                 |
| PROBLEM STATEMENT                                    |                 |
| METHODOLOGY                                          |                 |
| 3.1 Python code for Transient Radial Heat Conduction |                 |
| 3.1.1 Unsteady 1D Radial Heat Conduction Equation    |                 |
| 3.1.2 Finite Volume Method (FVM)                     |                 |
| 3.1.3 Tri-Diagonal Matrix Algorithm (TDMA)           |                 |
| 3.1.4 Python code                                    |                 |
| 3.2 Ansys APDL simulation                            |                 |
| RESULTS                                              |                 |
| 4.1 Ansys APDL                                       |                 |
| 4.2 Python code                                      |                 |
| 4.3 Verification                                     |                 |
| TECHNOLOGICAL ADVANCEMENTS                           |                 |
| CONCLUSION/ SUMMARY                                  |                 |

## TOPIC I:

### INTRODUCTION

#### 1.1 Convergent-Divergent Nozzle

The Convergent-Divergent Nozzle is also known as the de Laval nozzle after the Swedish inventor Gustat de Laval. In a CD nozzle a large fraction of the thermal energy of the gases in the chamber is converted into kinetic energy. The gas pressure and temperature drops drastically and the gas velocity can reach values in excess of two miles per second. This is reversible and essentially an isentropic flow process. This nozzle is designed to accelerate the subsonic flow in the converging part as it reaches the throat. At the nozzle throat, the critical velocity is sonic velocity. And then it accelerates out the diverging side of nozzle as supersonic flow or flow that is faster than the speed of sound. The speed of sound  $a_0$  in a material is given by  $a_0 = \sqrt{\gamma R T}$ , where  $\gamma$  is the ratio of the heat capacity at constant pressure,  $C_p$  to the heat capacity at constant volume,  $C_v$  and is also called specific heat ratio. The velocity of the flow divided by the speed of sounds in the flow is call Mach number and is

$$M = \frac{v}{\sqrt{\gamma R T}}$$

Gases and vapours are expanded in nozzles by providing a pressure ratio across them. The shape of the nozzle passage depends on the local Mach number with reference to the equation:

$$\frac{dA}{A} = \frac{dp}{\rho C^2} (1 - M^2)$$

Since the purpose of a nozzle is to accelerate the flow by providing a pressure drop,  $dp$  is negative. On the other hand, Diffusers are employed to obtain pressure rise for a given value of initial kinetic energy. The static pressure rise is at the loss of the deceleration of flow in the diffuser passage. Here  $dp$  is always positive. Another form of the above equation is,

$$\frac{-dA}{A} = \frac{dv}{v} (1 - M^2)$$

The equation shows that if the Mach number is greater than 1 and a positive area change in the nozzle indicates a positive velocity change. For Mach number less than 1 and positive area change, the velocity change is negative. This shows that why a converging nozzle speeds up the subsonic flow and the diverging nozzle speeds up the supersonic flow.

## PROBLEM STATEMENT

A convergent-divergent nozzle is consisting of the following dimensions :

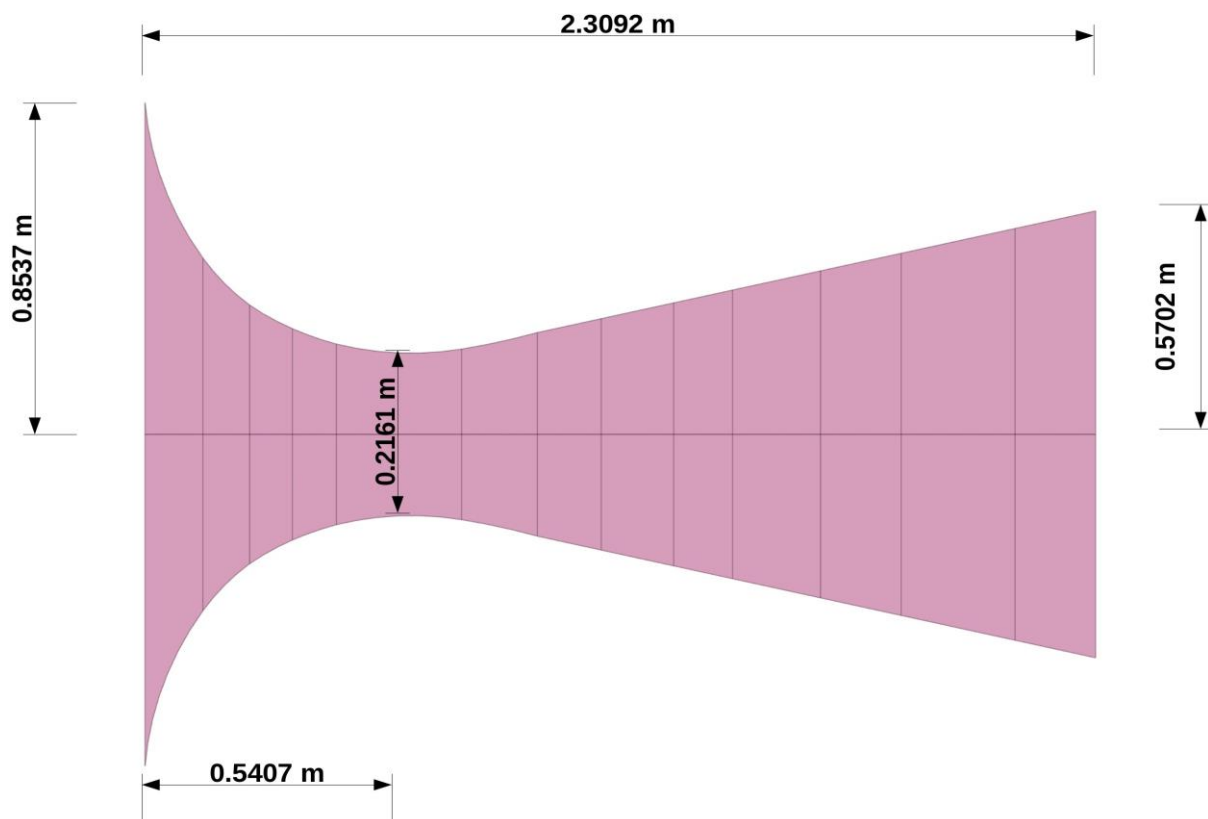
Inlet radius = 0.8537 m

Throat radius = 0.21616 m

Outlet radius = 0.575024 m

Length of the nozzle = 2.3092115 m

Distance of throat from inlet = 0.5407159 m



*fig 1 : Geometry and dimensions of the Convergent-Divergent nozzle*

Taking fluid flowing through the convergent-divergent nozzle as the perfect gas, the following properties need to be assigned for the inlet conditions of the convergent divergent nozzle :

Density of air =  $1.225 \text{ kg/m}^3$

Inlet pressure = 40 bar =  $40,00,000 \text{ N/m}^2$

Inlet temperature = 2800 K

Inlet velocity = 10 m/s

The wall of the CD nozzle is isothermal with wall temperature = 300K



Considering steady state flow inside the nozzle, variation of Mach number along the nozzle has been studied through CFD ++ software, after meshing the model in the Pointwise software.

Empirical relation of Area ratio and Mach number from the “isentropic relations” of convergent-divergent nozzle:

$$\frac{A}{A_{throat}} = \frac{1}{M} \left[ \frac{2}{\gamma + 1} \left( 1 + \frac{\gamma - 1}{2} \right) M^2 \right]^{\frac{\gamma + 1}{2(\gamma - 1)}}$$

Using this relationship, a Python code has been designed to solve for Mach number at every x coordinate in the CD nozzle, that is computing area ratio at every x location and solving the empirical equation.

The plot of Area ratio v/s Mach number (values taken from solving the empirical relationship) which gives the variation of Mach number along the varying area ratio of the nozzle (indirectly giving the varying Mach number along the x coordinate of the nozzle) has been verified with the ones obtained from the results of simulation of CFD++.

## METHODOLOGY

### 3.1 Python code for Area-Mach relationship

Area-Mach relationship for any points x and y within the nozzle, taken from the book “Rocket propulsion Elements – George P. Sutton, Oscar Biblarz”:

$$\frac{A_y}{A_x} = \frac{M_x}{M_y} \sqrt{\frac{\left(1 + \left[\frac{k-1}{2}\right] M_y^2\right)^{\frac{k+1}{k-1}}}{\left(1 + \left[\frac{k-1}{2}\right] M_x^2\right)}}$$

Considering x as the position at throat, then  $A_x = A_{throat}$ ,  $M_x = 1.0$ .

Simplifying the equation,

$$\frac{A}{A_{throat}} = \frac{1}{M} \left[ \frac{2}{\gamma + 1} \left( 1 + \frac{\gamma - 1}{2} M^2 \right) \right]^{\frac{\gamma + 1}{2(\gamma - 1)}}$$

Solving this equation for Mach number was done in an iterative method thereby converging towards the solution. This is done with the help of numerical methods.

Among all the numerical methods, “Bisection method” is quite suitable to solve the empirical equation.

#### 3.1.1 Bisection method

If the roots of a function  $f(x)$  lie in the domain  $[a, b]$ , then  $f(a) \times f(b) < 0$ , and vice versa.

Bisection is made along the domain in between a and b, i.e.  $\frac{a+b}{2}$ . If  $f(a) \times f\left(\frac{a+b}{2}\right) < 0$ , then the root lies in between a and  $\frac{a+b}{2}$ . And if  $f\left(\frac{a+b}{2}\right) \times f(b) < 0$ , then the root lies between  $\frac{a+b}{2}$  and b.

Now for equation (1), for subsonic region (from inlet to throat), value of Mach number lies in  $[0,1]$ . And for the supersonic region (throat to outlet) the upper bound of the value of Mach number is taken to be 50, hence the range being  $[1,50]$ .

#### 3.1.2 Nozzle geometry

Origin is taken to be at the inlet of the nozzle and axis of the nozzle, symmetric axis of the nozzle is taken as X axis and perpendicular to it directing towards the wall of the nozzle is taken as Y axis.

The equation of the CD nozzle found out from the coordinates taken from pointwise:

Converging region :

$$3.0128x^2 - 2.72256x + 0.82961 = 0$$

Diverging region :

$$0.030528x^2 + 0.12917x + 0.120136 = 0$$

The output of these equations are the corresponding y coordinates at a particular location x.

The values of y coordinates are the radius of the CD nozzle.

### 3.1.3 Representing Area-Mach relationship as a function

$$\text{Let, } f(M) = \frac{1}{M} \left[ \frac{2}{\gamma+1} \left( 1 + \frac{\gamma-1}{2} \right) M^2 \right]^{\frac{\gamma+1}{2(\gamma-1)}} - \frac{A}{A_{throat}}$$

In the range of guesses of Mach numbers, let the lower bound be  $m_l$  and the upper bound be  $m_u$ .

Now,  $f(m_l) = f_l$  and  $f(m_u) = f_u$ .

Bisection requires that these have opposite signs so that a root lies between them.

$$\text{Let, } f_{mid} = f\left(\frac{m_u+m_l}{2}\right)$$

If  $f_l \times f_u > 0$ , we made it return to  $M = 1.0$  as a safe fallback (since  $M=1$  at the throat) rather than proceeding with bad brackets. This checks the two starting guesses actually sandwich a root. If they don't, it safely returns  $M = 1.0$ .

### 3.1.4 Python code

**#Import libraries**

import numpy as np

import matplotlib.pyplot as plt

**#Initialisation**

gamma = 1.4

x\_throat = 0.5407159

y\_throat = 0.21616

N=10000

**#Nozzle geometry**

x\_conv = np.array([0.0,0.14079,0.25403,0.35822,0.464866,0.54715])

```

y_conv = np.array([0.8537,0.45364,0.332823,0.27188,0.23244,0.21616])
x_div =
np.array([0.5407159,0.768987,0.953015,1.10838,1.28423,1.42716,1.64099,1.83700,2.11320
2,2.3092115])
y_div =
np.array([0.21616,0.219436,0.261922,0.29779,0.33839,0.371389,0.42075,0.46600,0.52977,
0.575024])
coeffs_conv = np.polyfit(x_conv , y_conv , 2)
coeffs_div = np. polyfit(x_div , y_div , 2)
f_conv = np.poly1d(coeffs_conv)
f_div = np.poly1d(coeffs_div)
x_vals = np.linspace(x_conv[0],x_div[-1],N)
y_vals = np.where(x_vals<=x_throat , f_conv(x_vals) , f_div(x_vals))

```

#### #Computing Area ratio A/A\*

```

A_throat=np.pi*y_throat**2
A_vals=np.pi*y_vals**2
area_ratios = A_vals/A_throat

```

#### #Area ratio Mach relationship function

```

def area_mach_func(M, AR):
    t = (2/(gamma+1))*(1+0.5*(gamma-1)*M**2)
    exp = (gamma+1)/(2*(gamma-1))
    return ((1/M)*(t)**exp)-AR

```

#### #Solving for Mach number by bisection method

```

def solve_mach(AR, m_low, m_high, tol=1e-6, max_iter=1000):
    if abs(AR-1.0) == 0:
        return 1.0
    fl=area_mach_func(m_low, AR)
    fh=area_mach_func(m_high, AR)

    if fl*fh>0 :
        return 1.0

```

```

for _ in range (max_iter):
    m_mid=0.5*(m_low + m_high)
    fm=area_mach_func(m_mid, AR)
    if abs(fm) == 0:
        return m_mid

    if fl*fm < 0 :
        m_high,fh=m_mid,fm
    else :
        m_low,fl = m_mid,fm
    return 0.5*(m_low + m_high)

```

### #Solve for Mach number everywhere

```

mach_vals=np.empty_like(area_ratios)
for i,AR in enumerate (area_ratios):
    if x_vals[i] == x_throat :
        mach_vals[i] = 1
    if x_vals[i] <= x_throat :
        mach_vals[i] = solve_mach(AR,1e-6,1)
    else :
        mach_vals[i] = solve_mach(AR,1,50)
M_min=mach_vals.min()
M_max=mach_vals.max()

print(f'Minimum Mach number : {M_min:.6f}')
print(f'Maximum Mach number : {M_max:.6f}')

```

### #plot Area Ratio vs Mach number

```

plt.figure(figsize=(10,8))
plt.plot(area_ratios,mach_vals, lw=2)
plt.ylabel ('Mach number (M)')
plt.xlabel ('Area ratio (A/A*)')
plt.axvline(1,color='gray',ls='--',label='M=1 at throat')
plt.axhline(1,color='gray',ls='--')

```

```
plt.title('Mach number v/s Area ratio')  
plt.legend()  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```

### 3.2 CFD++ simulation

After designing the structure of the nozzle in pointwise according to the dimensions mentioned, the following measures were taken into account before meshing:

- At the inlet of the nozzle, an arc perpendicular to the wall of the nozzle was made to make the maximum included angle around  $90^\circ$  for better mesh.
- Partitions were made within the nozzle for better study of grids at narrow region and at the outlet of the nozzle.

Meshing process:

- Imported the database data.
- Selected the CAE solver and dimension
- Created grid entities (Connectors, Domains)
- Refined or smoothed the mesh as necessary.
- Set CAE Boundary conditions.
- Exported work in the appropriate CAE format.
- Saved the project.

Boundary conditions named in Pointwise:

- Wall
- Symmetry
- Outlet
- Inlet

Exporting the file to CFD++ after meshing in Pointwise, the following are the setup synopsis:

**CFD Problem Definition:**

Fluid type : Gas

Species content : Single Species

Compressible flow : Yes

Gas type : Perfect gas

Flow speed : Mostly above Mach 0.5

Viscous flow : Yes

Turbulent flow : No

Radiation Heat Transfer : No

Simulation type : Steady State

**Additional Specifications:**

2D case type : 2D Axysymmetric

**Equation properties:**

No change required

(Compressible perfect gas Navier Stokes Equations will be solved)

**Physics source terms:**

No change required

**Reference quantities:**

No change required

**Fluid properties – Primary Phase:**

No change required (fluid is air)

**Initial Conditions:**

Initialization types : Entire Domain

Variables to use : Density-based

Static Pressure : 101325.0 Pa

Density : 1.225 kg/m<sup>3</sup>

X-Velocity : 10.0 m/s

Y-Velocity : 0.0m/s

Z-Velocity : 0.0m/s

**Boundary Conditions:**

- 1 – **Outlet** : Open  
Boundary Condition Sub-group : Outflow only  
Impose a condition at boundary : No  
Applied Condition type : Supersonic outflow
- 2 – **Symmetry** : Symmetry  
Type of symmetry : Symmetry
- 3 – **Wall** : Wall  
Wall type : Viscous  
Wall Heat Transfer type : Isothermal  
Wall Temperature : 300 K
- 4 – **Inlet** : Open  
Boundary Condition Sub-group : Inflow only  
Information type : Pressure and Temperature  
Applied Condition type : Stagnation Pressure and Temperature  
Stagnation Pressure = 40,00,000 N/m<sup>2</sup>  
Stagnation Temperature = 2800 K

**Help set numbers:**

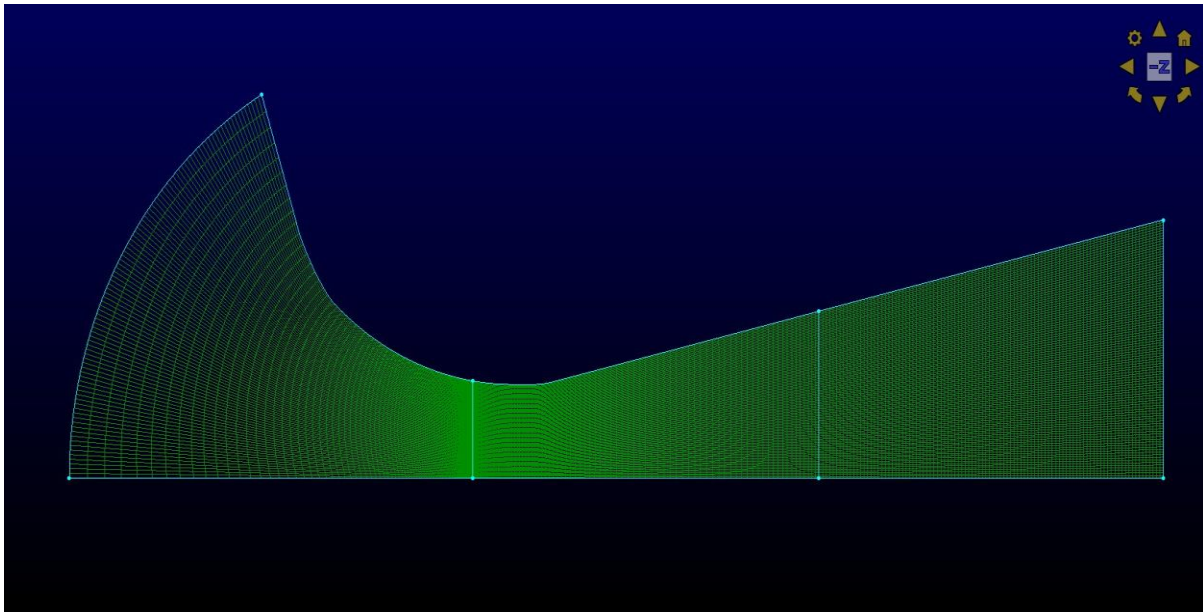
Select : Set numerical parameters based on physical regime

Flow regime : Low supersonic



## RESULTS

### 4.1 Meshing



*fig 2 : Meshed Convergent-Divergent Nozzle in Pointwise*

Number of points each connector is divided into for grid formation = 100

Pointwise offers various mathematical metrics to evaluate mesh quality. These are examined below:

#### **Jacobian:**

This measures the volume or area change caused by the mesh transformation. A positive Jacobian is desirable indicating a valid element.

*Minimum value : 0.000427292867*

*Maximum value : 2.1573797e-06*

#### **Area Ratio:**

This compares the area of an element to the ideal element area within the mesh. Area ratio close to 1 indicates a well-proportioned element.

*Minimum value : 1.0017391*

*Maximum value : 1.1200869*

**Aspect Ratio:**

It is the ratio of the longest to the shortest side (or edge) of an element. It measures how stretched or elongated an element is. High aspect ratios are often used in boundary layers, but too high can degrade the accuracy. Ideal value is 1 for isotropic cells.

*Minimum value : 1.0000536*

*Maximum value : 5.8218449*

**Minimum Included Angle:**

It refers to the smallest angle between adjacent faces of an element. A well shaped element will have angles close to  $90^\circ$  for quadrilaterals in 2D.

*Minimum value : 73.066735*

*Maximum value : 89.991045*

**Maximum Included Angle:**

It is the largest internal angle in an element. Very large angles (approaching  $180^\circ$ ) can indicate degenerate elements. Ideal value is less than  $120^\circ$  for triangles and less than  $150^\circ$  for quads.

*Minimum value : 90.008955*

*Maximum value : 107.62851*

**Skewness Equi angle:**

It tells how far the angles in a mesh element deviate from their ideal or “equiangular” value. The ideal element is one in which all internal angles are equal. Ideal value should be 0 for a perfect element and 1 for a very bad element.

*Minimum value : 0.00013577265*

*Maximum value : 0.19587232*

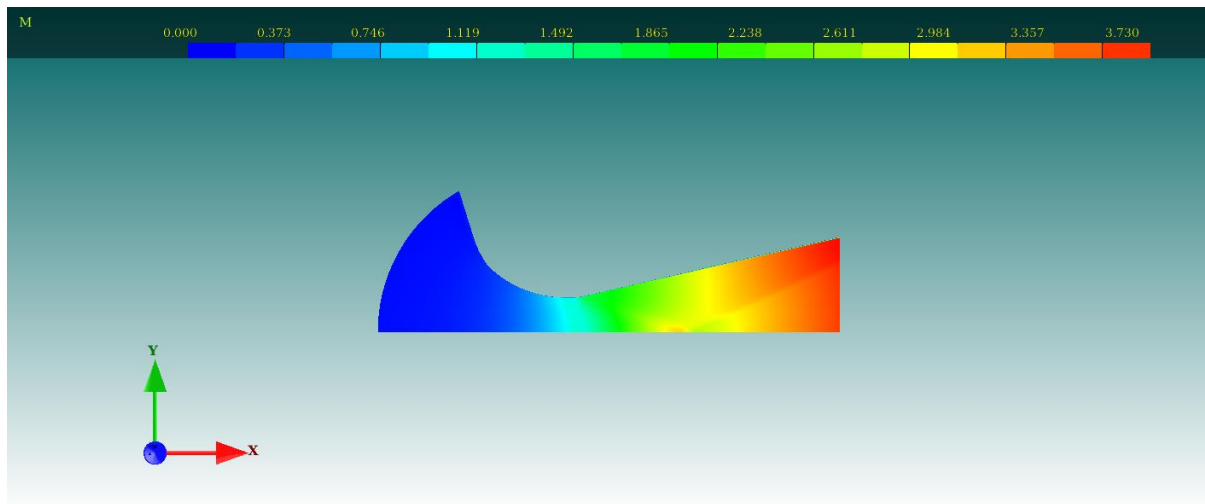
## 4.2 CFD++

After simulating the flow through nozzle in CFD++, the following results were observed:

### Mach number:

Maximum value : 3.72968578339

Minimum value : 0.0

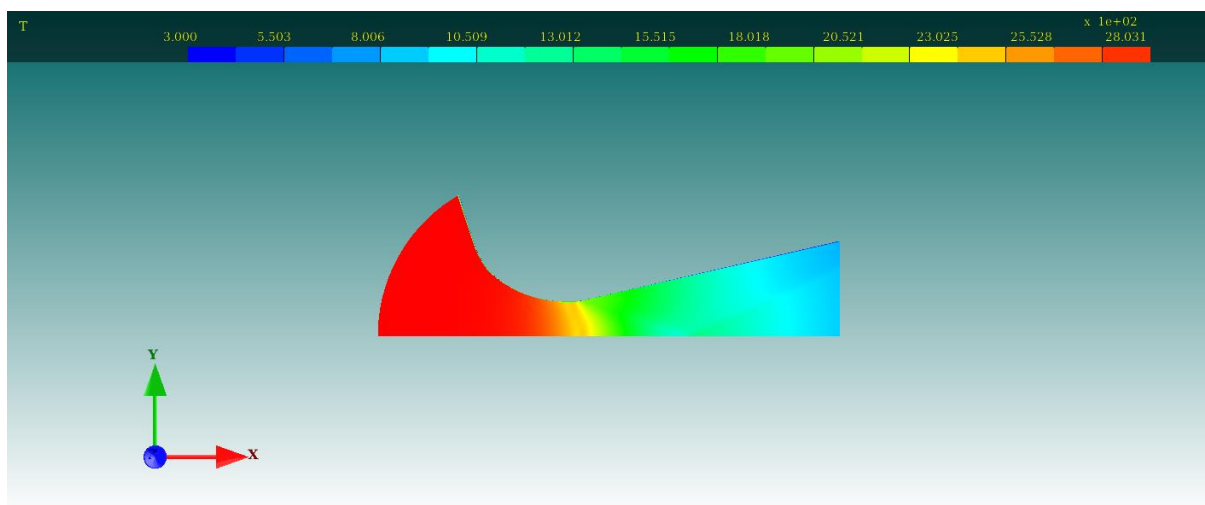


*fig 3 : Contour plot of Mach number along CD Nozzle*

### Temperature:

Maximum value : 2803.0690918 K

Minimum value : 300.0 K

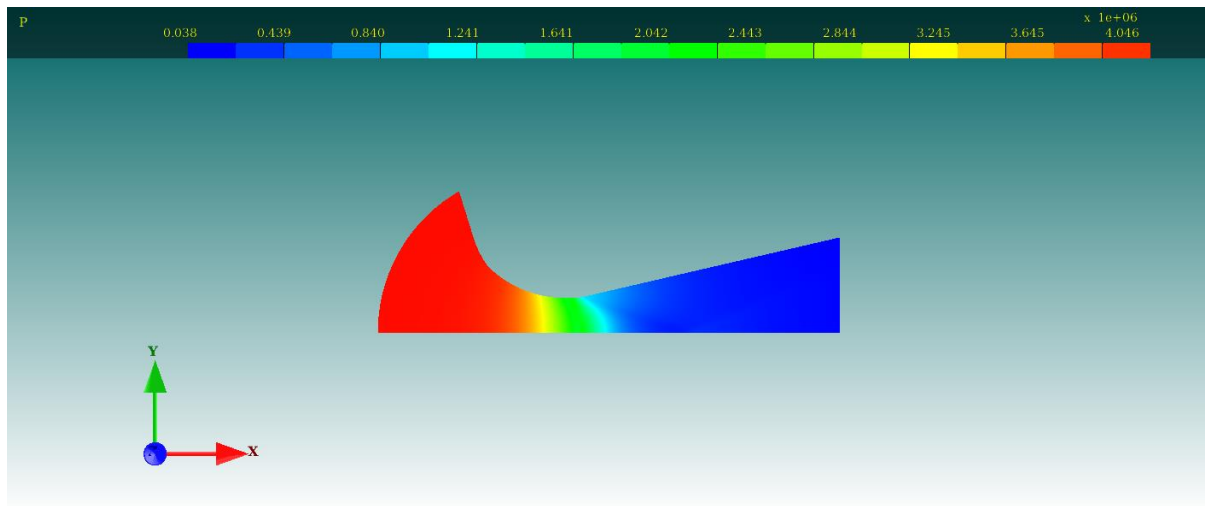


*fig 4 : Contour plot of Temperature along the CD Nozzle*

**Pressure:**

Maximum value : 4046295.0 N/m<sup>2</sup>

Minimum value : 38248.2265625 N/m<sup>2</sup>

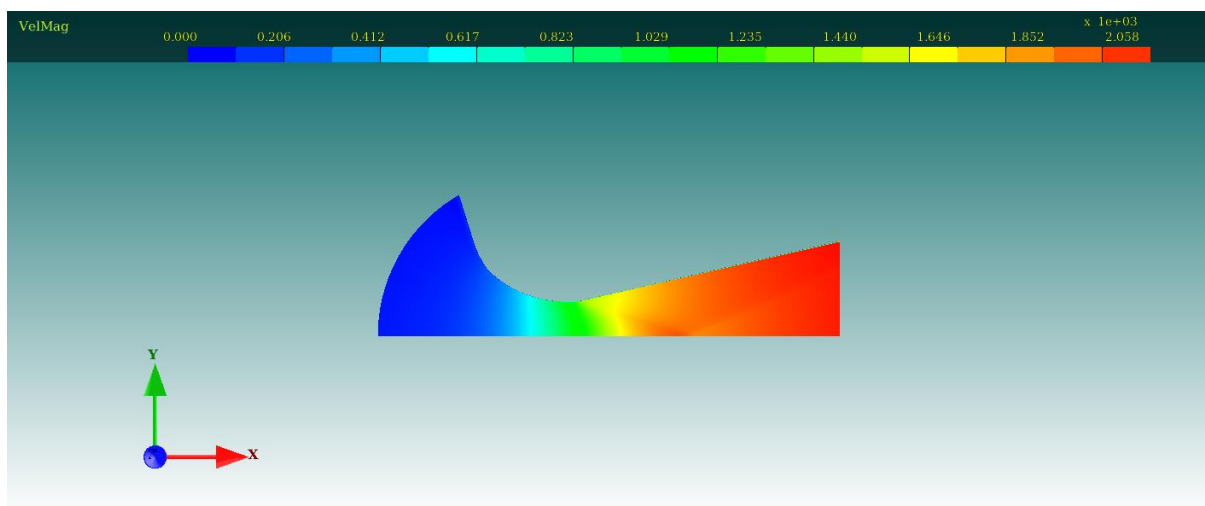


*fig 5 : Contour plot of Pressure along the CD Nozzle*

**Velocity Magnitude:**

Maximum value : 2057.55639648 m/s

Minimum value : 0.0 m/s



*fig 6 : Contour plot of Velocity magnitude along the CD nozzle*

Following is the result obtained from the code of “Finding Mach number from empirical relationship of Area ratio and Mach number”:

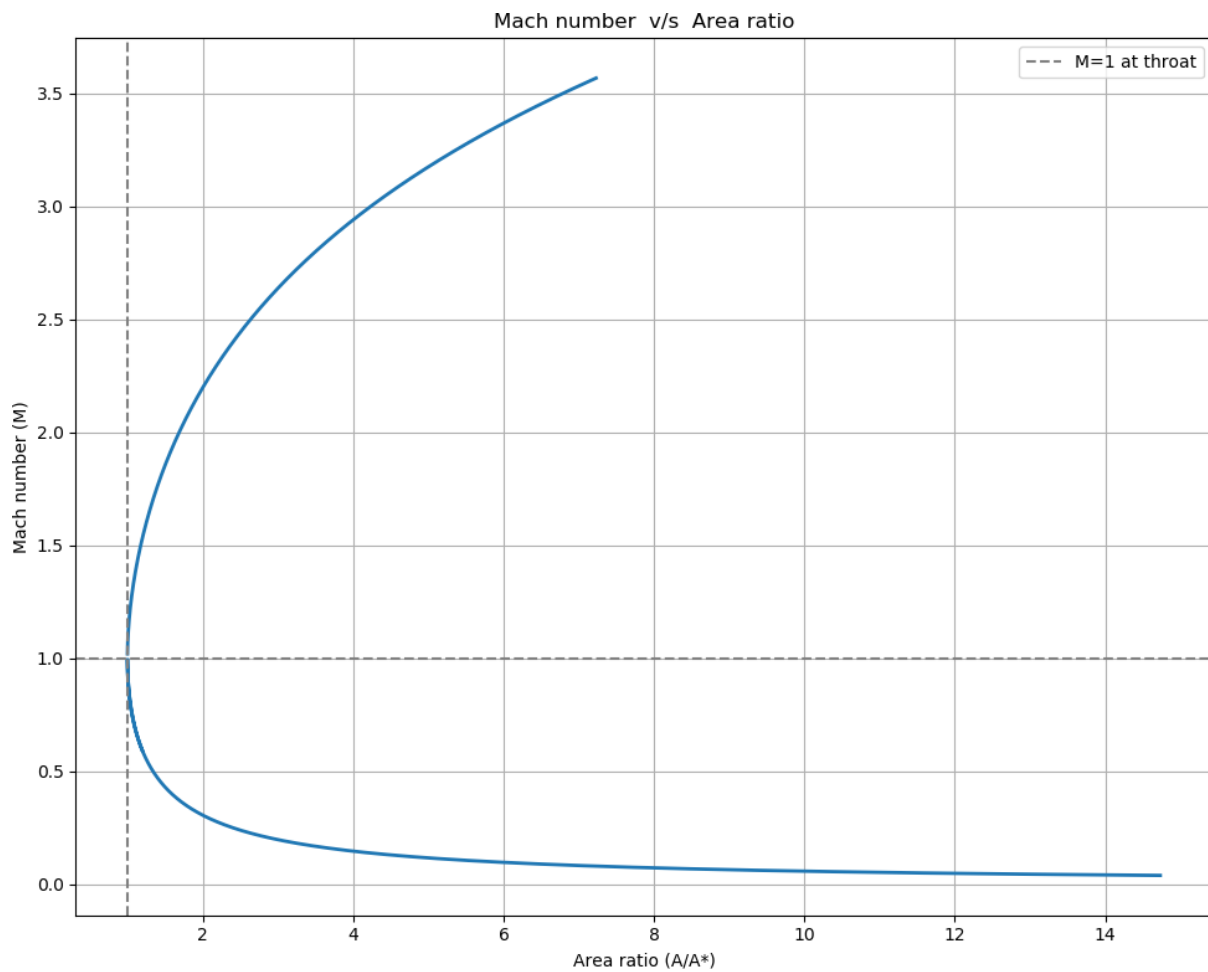


fig 7 : Plot of Mach number v/s Area Ratio of the CD Nozzle (obtained from Python code)

Minimum Mach number : 0.039324

Maximum Mach number : 3.567562

## C-D NOZZLE IN ROCKET PROPULSION

A convergent–divergent (CD) nozzle is a critical component in rocket propulsion systems, designed to optimize the conversion of thermal energy from combustion gases into directed kinetic energy, thereby producing thrust. The working principle of a CD nozzle is based on compressible fluid dynamics and the behaviour of gases at different velocities relative to the speed of sound.

In the convergent section, the nozzle narrows, causing the pressure and temperature of the exhaust gases to increase as the flow velocity rises. At the narrowest point, known as the throat, the gas velocity reaches Mach 1 (sonic speed). This throat condition is essential, as it marks the transition between subsonic and supersonic flow regimes. Beyond the throat, the nozzle diverges, allowing the gases to expand rapidly. Due to the pressure differential between the combustion chamber and the external atmosphere, the gas accelerates further, reaching supersonic speeds in the divergent section.

This expansion process maximizes the conversion of internal energy into kinetic energy, producing a high-velocity exhaust jet. The resulting reaction force, in accordance with Newton's third law of motion, propels the rocket forward. The thrust generated is directly proportional to both the mass flow rate of the gases and their exit velocity. CD nozzles are thus carefully designed to balance these parameters while minimizing energy losses.

Additionally, the effectiveness of a CD nozzle depends on external pressure conditions. At sea level, atmospheric pressure influences the degree of expansion, while in the vacuum of space, the nozzle operates more efficiently due to the absence of opposing pressure. This explains why rockets are often equipped with longer divergent sections for upper stages, ensuring optimal expansion and maximum thrust in low-pressure environments.

In summary, CD nozzles are indispensable in modern rocketry because they enable controlled acceleration of exhaust gases from subsonic to supersonic speeds, significantly improving thrust efficiency. Their geometry ensures stable flow conditions, reduces losses, and provides the high exhaust velocities necessary for overcoming Earth's gravity and enabling space travel.

## CONCLUSION/ SUMMARY

The numerical investigation of the convergent–divergent nozzle successfully demonstrated the effectiveness of combining high-fidelity CFD++ simulations with a Python-based empirical solver. The CFD++ analysis accurately captured the essential flow characteristics—subsonic acceleration in the converging region, sonic transition at the throat, and supersonic expansion in the diverging section—while providing detailed variations of Mach number, pressure, temperature, and velocity. These results reflect the capability of advanced CFD tools to model real operating conditions with high precision.

The Python solver, developed using the quasi one-dimensional area–Mach number relationship and solved through the bisection method, proved to be a reliable and cost-effective computational approach. Its modular design allowed user-defined inputs, rapid sensitivity analysis, and preliminary design optimization. A close comparison between the two methods revealed strong agreement, with the CFD++ predicting an exit Mach number of 3.7296 and the Python solver estimating 3.5676, corresponding to a deviation of only 4.5%. This validates the robustness of the empirical solver and demonstrates its applicability for preliminary analysis and cross-verification of CFD outputs.

Additionally, the study emphasized the importance of advanced meshing strategies in ensuring solution accuracy and stability. Grid refinement in critical regions such as the throat and outlet, along with the evaluation of mesh-quality parameters including Jacobian, aspect ratio, and skewness, contributed to the reliability of the CFD results. This integration of high-quality meshing with strong numerical methods highlights the technological advancements achieved in nozzle flow simulations.

In summary, the work illustrates how modern computational tools, when coupled with classical numerical techniques, provide a balanced framework for nozzle performance prediction. While CFD++ delivers high-fidelity insights into complex flow physics, the Python solver enables rapid and flexible evaluations at significantly lower computational cost. Together, they form a powerful methodology for nozzle design, validation, and optimization. Looking ahead, this dual-method approach holds strong potential for wider application in rocket propulsion, aerospace engineering, and other domains involving high-speed compressible flows, thereby contributing to innovation and improved design practices in future propulsion technologies.

## TOPIC II:

### INTRODUCTION

#### 1.1 Modes of Heat Transfer

When two points in a medium are at different temperatures, then heat flows from a point of higher temperature to a point of lower temperature.

There are three modes of heat transfer: Conduction, Convection and Radiation.

**Conduction:** It is a mechanism of heat propagation from a region of higher temperature to a region of lower temperature within a medium or between different mediums in direct physical contact. The conduction of heat is either due to Free electrons or due to lattice crystal vibrations. The governing law of conduction is Fourier law of heat conduction,  $\frac{Q}{A} \propto \left(\frac{-dT}{dx}\right)$ . The constant of proportionality,  $k$  is called thermal conductivity which indicates the rate at which heat energy is transferred through a medium by conduction process.

**Convection:** It is the process of energy transport affected by the circulation or mixing of a fluid medium. It is possible only in fluid medium and is directly linked with transport of medium itself. The governing law is Newton's law of cooling,  $\frac{Q}{A} \propto (T_s - T_\infty)$ . Proportionality constant is heat transfer coefficient,  $h$ . When the circulation of the fluid medium is caused by buoyancy effects (due to difference in densities), it is called Natural Convection. If this motion of fluid is produced by some super imposed velocity field, it is called Forced Convection.

**Radiation:** It is transmission of heat in the form of radiant energy or wave motion from one body to another across an intervening space. An intervening medium is not even necessary and the radiation can be affected through vacuum also; in fact, more effective in vacuum only.

Mechanism of radiation is divided into three phases:

- Conversion of thermal energy of the hot source into electromagnetic waves.
- Passage of wave motion through intervening medium.
- Transformation of waves into heat

For steady state heat conduction, the main assumption is that temperature is constant over a period of time. But in reality, it is not possible to maintain constant temperature. This is called Unsteady/Transient state heat conduction which is of time dependent. Change in temperature during unsteady state may follow a periodic or non-periodic variation. In periodic variation, the temperature changes in repeated cycles and the conditions get repeated after some fixed time interval. On the other hand, in non-periodic, the variation in temperature is neither according to any definite pattern nor is in repeated cycles.



## PROBLEM STATEMENT

The cross section of nozzle is annular. Hence at a particular cross section, the temperature variation along the cross section of the wall in 1D has been found out using 1D unsteady radial heat diffusion equation.

1D Unsteady radial heat diffusion equation:

$$\rho C_p \frac{dT}{dr} = \frac{1}{r} \frac{d}{dr} \left( kr \frac{dT}{dr} \right)$$

In order to find temperature distribution in the body, a Python code has been developed to solve the unsteady radial heat diffusion equation using Finite Volume Method (FVM). Implicit scheme has been implemented which is conditionally stable scheme.

Inner radius,  $r_{\text{inner}} = 2.005 \text{ m}$

Outer Radius,  $r_{\text{outer}} = 2.01 \text{ m}$

The material properties are as follows :

Density,  $\rho = 1400 \text{ kg/m}^3$

Specific Heat,  $C_p = 1000 \text{ J/kgK}$

Conductivity,  $k = 0.1 \text{ W/mK}$

Uniform Temperature,  $T_0 = 300 \text{ K}$

For boundary conditions, constant flux at inner and outer part of the slice need to be considered with the following values:

Flux on inner wall,  $q_{\text{inner}} = 20,000 \text{ W/m}^2$

Flux on outer wall,  $q_{\text{outer}} = 0 \text{ W/m}^2$

The plot of the Temperature at each location of  $r$  (for each cell) has been studied.

In order to verify the Python code, verification has been done in Ansys APDL for same conditions.

## METHODOLOGY

### 3.1 Python code for Transient Radial Heat Conduction

#### 3.1.1 Unsteady 1D Radial Heat Conduction Equation

The Partial Differential Equation (PDE) for 1D unsteady radial heat conduction equation is:

$$\rho C_p \frac{dT}{dt} = \frac{1}{r} \frac{d}{dr} \left( kr \frac{dT}{dr} \right)$$

#### 3.1.2 Finite Volume Method (FVM)

Taking a cell in which the green area is the control volume,

P represents the present cell

N represents the North cell, and  $q_N$  represents the flux from North cell

S represents the South cell, and  $q_S$  represents the flux from South cell

W represents the West cell, and  $q_W$  represents the flux from West cell

E represents the East cell, and  $q_E$  represents the flux from East cell

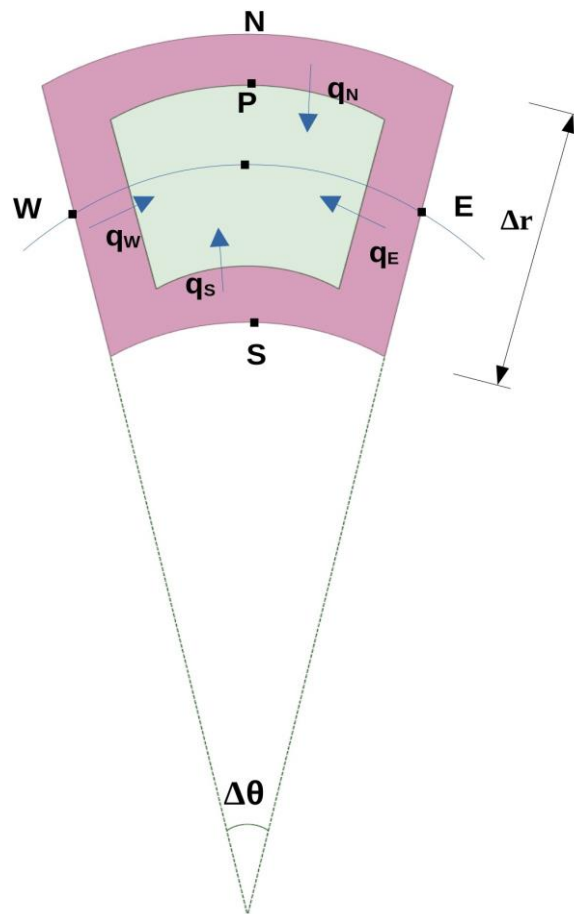


fig 8 : Visualisation of Control Volume in a cell of the annular section

After discretizing the equation using implicit scheme and TDMA:

$$\rho C_p (T_i^{n+1} - T_i^n) = \Delta t [a_{S,i} (T_{i-1}^{n+1} - T_i^{n+1}) + a_{N,i} (T_{i+1}^{n+1} - T_i^{n+1})] + \Delta t [\text{boundary fluxes}]$$

Simplifying,

$$a_i T_{i-1}^{n+1} + b_i T_i^{n+1} + c_i T_{i+1}^{n+1} = d_i$$

Where,

Lower-diagonal,  $a_i = \frac{-\Delta t a_{S,i}}{\rho C_p V_i}$ , with  $a_1 = 0$

Main-diagonal,  $b_i = 1 + \frac{\Delta t [a_{S,i+1} + a_{N,i}]}{\rho C_p V_i}$

Upper-diagonal,  $c_i = \frac{-\Delta t a_{N,i}}{\rho C_p V_i}$ , with  $c_n = 0$

Right hand side,  $d_i = T_i^n + (\text{any added boundary flux normalised by } \rho C_p V_i)$

Thus, forming the matrix equation:

$$AX = B$$

Where,

A = Tridiagonal matrix containing the coefficients of discretized equations (i.e.  $a_i, b_i, c_i, d_i$ )

X = Matrix of unknown temperature variables

B = Matrix of constants in the equation

For each discretized cell:

Cell width,  $\Delta r = \frac{r_{outer} - r_{inner}}{100}$

Area of each cell wall,  $A = r_{edge}(\Delta\theta)L$  [where,  $r_{edge}$  = radius of cell wall]

Radius of cell wall,  $r_{edge} = \frac{r_{i-1} + r_i}{2}$

Volume of each cell,  $V_i = \int_{r_i}^{r_{i+1}} r(\Delta\theta)L dr$

$$V_i = \left( \frac{r_{i+1}^2 - r_i^2}{2} \right) (\Delta\theta)L$$

### 3.1.3 Tri-Diagonal Matrix Algorithm (TDMA)

Solving TDMA:

Eliminating temperature variable of the previous node through forward elimination: (lower diagonal elements will be eliminated)

for  $i = 1$  (first cell),  $b_1 T_1^{n+1} + c_1 T_2^{n+1} = d_1$

$$T_1^{n+1} + c'_1 T_2^{n+1} = d'_1 \quad \left[ \text{where, } c'_1 = \frac{c_1}{b_1} \text{ and } d'_1 = \frac{d_1}{b_1} \right]$$

$$T_1^{n+1} = d'_1 - c'_1 T_2^{n+1} \quad \text{----- (1)}$$

for i = 2 (second cell),  $a_2 T_1^{n+1} + b_2 T_2^{n+1} + c_2 T_3^{n+1} = d_2$

$$a_2(d'_1 - c'_1 T_2^{n+1}) + b_2 T_2^{n+1} + c_2 T_3^{n+1} = d_2 \quad [\text{from (1)}]$$

$$T_2^{n+1}(b_2 - a_2 c'_1) + c_2 T_3^{n+1} = d_2 - a_2 d'_1$$

and so on.

Hence the new diagonal term will be,  $\tilde{b}'_i = b_i - a_i c'_{i+1}$

Modified upper diagonal will be,  $\tilde{c}'_i = \frac{c_i}{b_i}$

Modified right hand side ( matrix of constants ) will be,  $\tilde{d}'_i = \frac{d'_i - a_i d'_{i-1}}{\tilde{b}_i}$

Back substitution :

$$\text{for } i = N, \quad T_N^{n+1} = d'_N \quad \text{----- (2)}$$

$$\text{for } i = N-1, \quad T_{N-1}^{n+1} + c'_{N-1} T_N^{n+1} = d'_{N-1}$$

$$T_N^{n+1} = d'_{N-1} - c'_{N-1} d'_N$$

and so on.

By doing this, the matrix equation will be solved giving the temperature value at each node.

### 3.1.4 Python code:

**#Importing libraries**

import numpy as np

import matplotlib.pyplot as plt

**#Initialisation**

r\_inner = 2.005

r\_outer = 2.01

k = 0.1 #W/mK

Cp = 1000.0 #J/kgK

rho = 1400.0 #kg/m^3

T0 = 300.0 #K

```

d_theta = 0.1 #rad
L=1.0 #m
Nr=100

#Providing flux at inner and outer wall
q_inner = +20000 #W/m^2
q_outer = +00000 #W/m^2

#Radius of cells
r=np.linspace(r_inner,r_outer,Nr)
dr=r[1]-r[0]

#Radius of the wall of the cell
r_edge = np.empty(Nr+1)
r_edge[0]=r_inner
r_edge[-1] = r_outer
for j in range (1,Nr) :
    r_edge[j] = 0.5*(r[j-1]+r[j])

#Area of the wall of the cell
A=r_edge*d_theta*L

#Volume of each cell
Vol=np.zeros(Nr)
for i in range (Nr):
    Vol[i] = d_theta*L*(r_edge[i+1]**2-r_edge[i]**2)/2
    a_S=np.zeros(Nr)
    a_N=np.zeros(Nr)

for i in range (1,Nr):
    a_S[i]=k*A[i]/dr
for i in range (Nr-1):
    a_N[i]=k*A[i+1]/dr
a_P = a_S + a_N

```

**#Initialisation of Time #Nt=1600**

```
time_total = 30 #s #dt = time_total/Nt
```

```
dt = 0.1
```

```
Nt = 300
```

**#Defining inner and outer cell Temperature array**

```
time = np.zeros(Nt+1)
```

```
T_inner = np.zeros(Nt+1)
```

```
T_outer = np.zeros(Nt+1)
```

```
time_step = 300
```

```
T_time_step = np.zeros(Nr)
```

```
T_new = np.zeros(Nr)
```

**#Function for solving Tri-diagonal matrix def TDMA(a,b,c,d):**

```
def TDMA(a,b,c,d):
```

```
    N = len(b)
```

```
    c_prime = np.zeros(N-1)
```

```
    d_prime = np.zeros(N)
```

```
    T_new = np.zeros(N)
```

```
    c_prime[0] = c[0]/b[0]
```

```
    d_prime[0] = d[0]/b[0]
```

```
    for i in range (1,N-1):
```

```
        c_prime[i] = c[i]/(b[i]-a[i]*c_prime[i-1])
```

```
        d_prime[i] = (d[i]-a[i]*d_prime[i-1])/(b[i]-a[i]*c_prime[i-1])
```

```
    d_prime[-1] = (d[-1]-a[-1]*d_prime[-2])/(b[-1]-a[-1]*c_prime[-2])
```

```
    T_new[-1] = d_prime[-1]
```

```
    for i in range (N-2,-1,-1) :
```

```
        T_new[i] = d_prime[i]-c_prime[i]*T_new[i+1]
```

```
    return T_new
```

```
T = np.full(Nr,T0,dtype=float)
```

**#Solving for Temperature from the matrix equation,  $AX = B$**

for step in range (1,Nt+1):

$T_{old} = T.copy()$

$\alpha_S = dt*a_S/(Vol*rho*Cp)$

$\alpha_N = dt*a_N/(Vol*rho*Cp)$

$a = -\alpha_S.copy()$

$c = -\alpha_N.copy()$

$a[0] = 0.0$

$c[-1] = 0.0$

$b = 1.0 + \alpha_S + \alpha_N$

$T_{new} = T_{old}.copy()$

$T_{new}[0] = T_{new}[0] + ((dt*q_{inner}*A[0])/(Vol[0]*rho*Cp))$

$T_{new}[-1] = T_{new}[-1] + ((dt*q_{outer}*A[-1])/(Vol[-1]*rho*Cp))$

$T = TDMA(a,b,c,T_{new})$

$time[0] = 0.0$

$T_{inner}[0] = T_0$

$T_{outer}[0] = T_0$

$time[step] = step*dt$

$T_{inner}[step] = T[0]$

$T_{outer}[step] = T[-1]$

**#Plotting**

if step==300:

**#Temperature variation with respect to radius at the final time step**

$plt.figure(figsize = (8,6))$

$plt.plot(r,T,label=f'step\{step\} \ t = \{step*dt:.3f\} s')$

$plt.title('T(r)')$

$plt.xlabel('Radius r')$

$plt.ylabel('Temperature T')$

$plt.grid(True)$

```
plt.legend()
```

```
plt.show()
```

**#Temperature variation in first cell (inner cell) with respect to time**

```
plt.figure(figsize = (8,6))
```

```
plt.plot(time,T_inner)
```

```
plt.title('T_inner T (t)')
```

```
plt.xlabel('Time t (s)')
```

```
plt.ylabel('Temperature T (K)')
```

```
plt.grid(True)
```

```
plt.show()
```

**#Temperature variation in first cell (inner cell) with respect to time**

```
plt.figure(figsize = (8,6))
```

```
plt.plot(time,T_outer)
```

```
plt.title('T_outer T (t)')
```

```
plt.xlabel('Time t (s) ')
```

```
plt.ylabel('Temperature T (K)')
```

```
plt.grid(True)
```

```
plt.show()
```



### 3.2 Ansys APDL simulation

For the simulation, the selected element type was “Quad 4 Node 55,” a two-dimensional element consisting of four nodes.

The material properties were defined as follows:

- Specific heat,  $C = 1000 \text{ J/kg}\cdot\text{K}$
- Density,  $\rho = 1400 \text{ kg/m}^3$
- Thermal conductivity,  $k = 0.1 \text{ W/m}\cdot\text{K}$

A two-dimensional annular section with an included angle of  $6^\circ$  (0.1 rad) was created, and the aforementioned material properties were assigned. The values of material properties and geometric dimensions used in ANSYS APDL were kept identical to those specified in the Python code to ensure consistency in comparison.

For meshing, the element edge length was set to 0.5 mm in order to achieve close agreement between the ANSYS and Python simulation results.

In the loading conditions, a uniform initial temperature of 300 K was applied to the entire area. Additionally, a heat flux of  $20,000 \text{ W/m}^2$  was imposed on the inner wall.

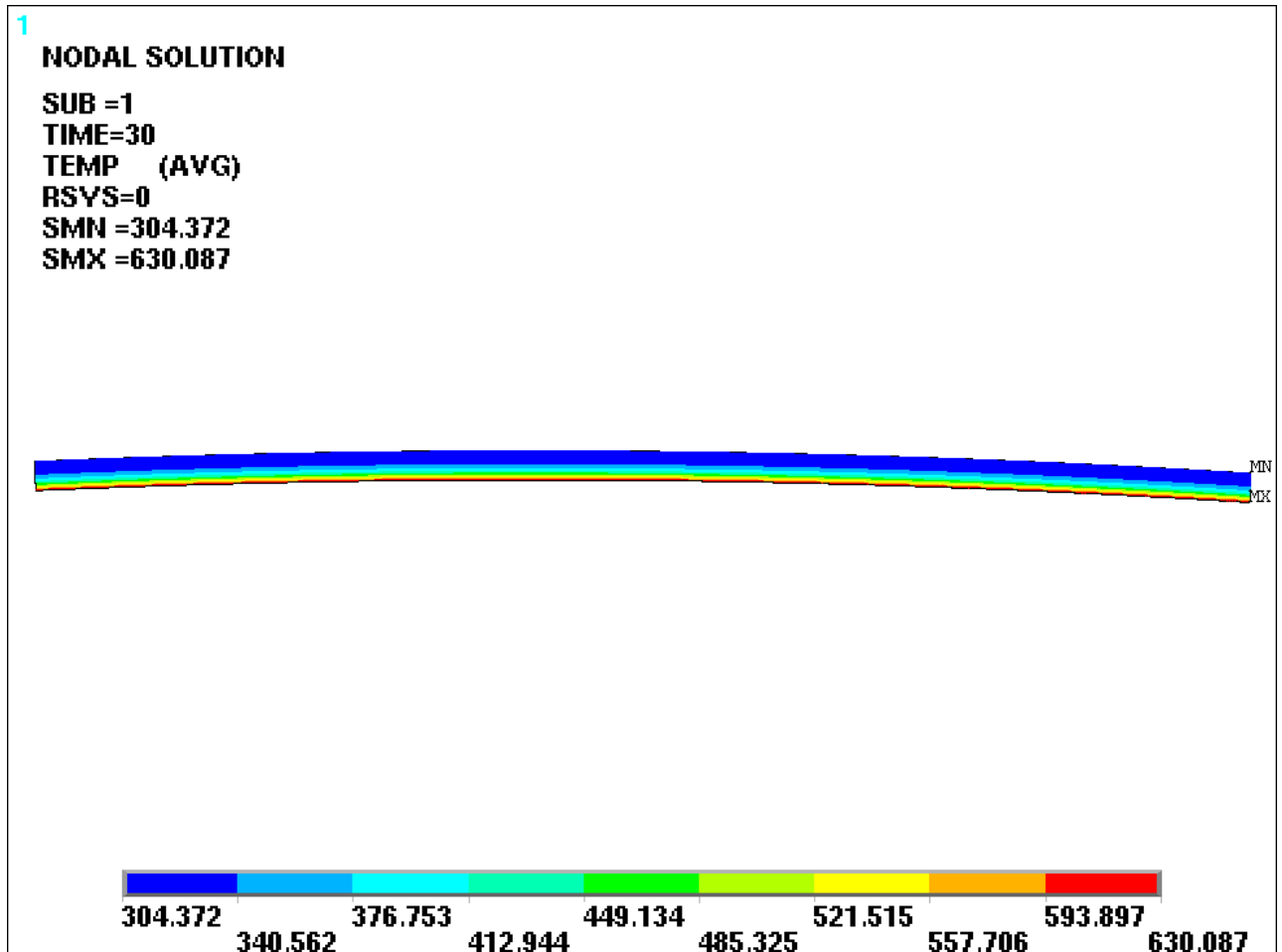
The transient thermal analysis was carried out with a time step size of 0.1 s, and the total simulation time was chosen to be 30 s.

## RESULTS

### 4.1 Ansys APDL

Simulation was done for the annular section of inner radius 2.005m and outer radius 2.01m with flux at inner wall as  $20,000 \text{ W/m}^2$  and flux at outer wall as  $0 \text{ W/m}^2$ .

The contour plot of the temperature after 30 seconds was obtained as shown below:



*fig 9 : Countour plot of Temperature of the annular section taken from Ansys APDL*

The maximum temperature was found out at the first cell (inner cell) with value 630.087 K.

The minimum temperature was found out the last cell (outer cell) with value 304.372 K.

This is because of the significantly higher value of flux provided at the inner wall ( $q_{\text{inner}} = 20,000 \text{ W/m}^2$ ) compared to the flux provided at the outer wall ( $q_{\text{outer}} = 0 \text{ W/m}^2$ ).

Temperature variation of the first cell (inner cell) with respect to time :

Resnick, M. – *Scratch Programming for Kids* (for block-based intro in Classes 6–8).

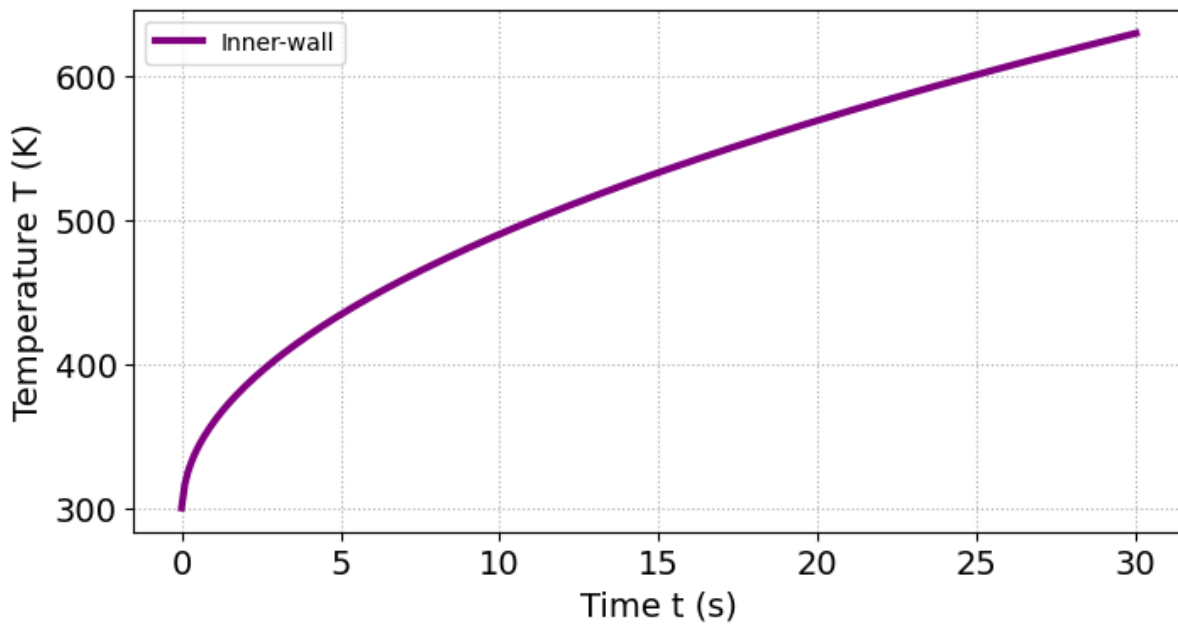


fig 10 : Plot of Temperature v/s time of the inner wall of the annular section (from Ansys APDL)

Temperature variation of the last cell (outer cell) with respect to time :

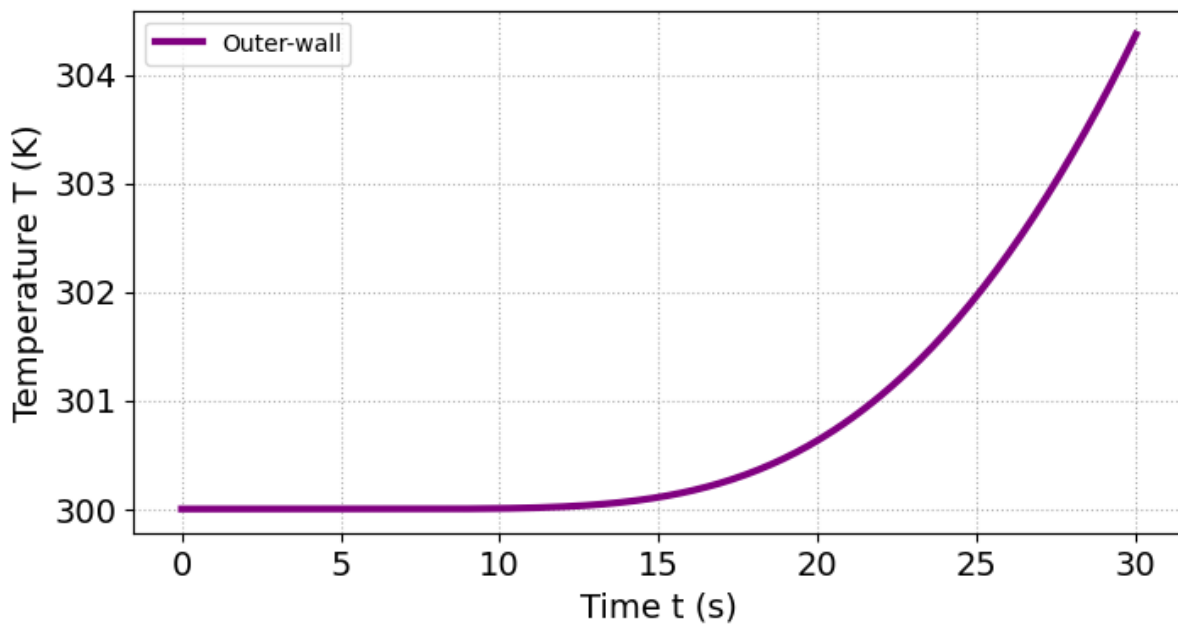
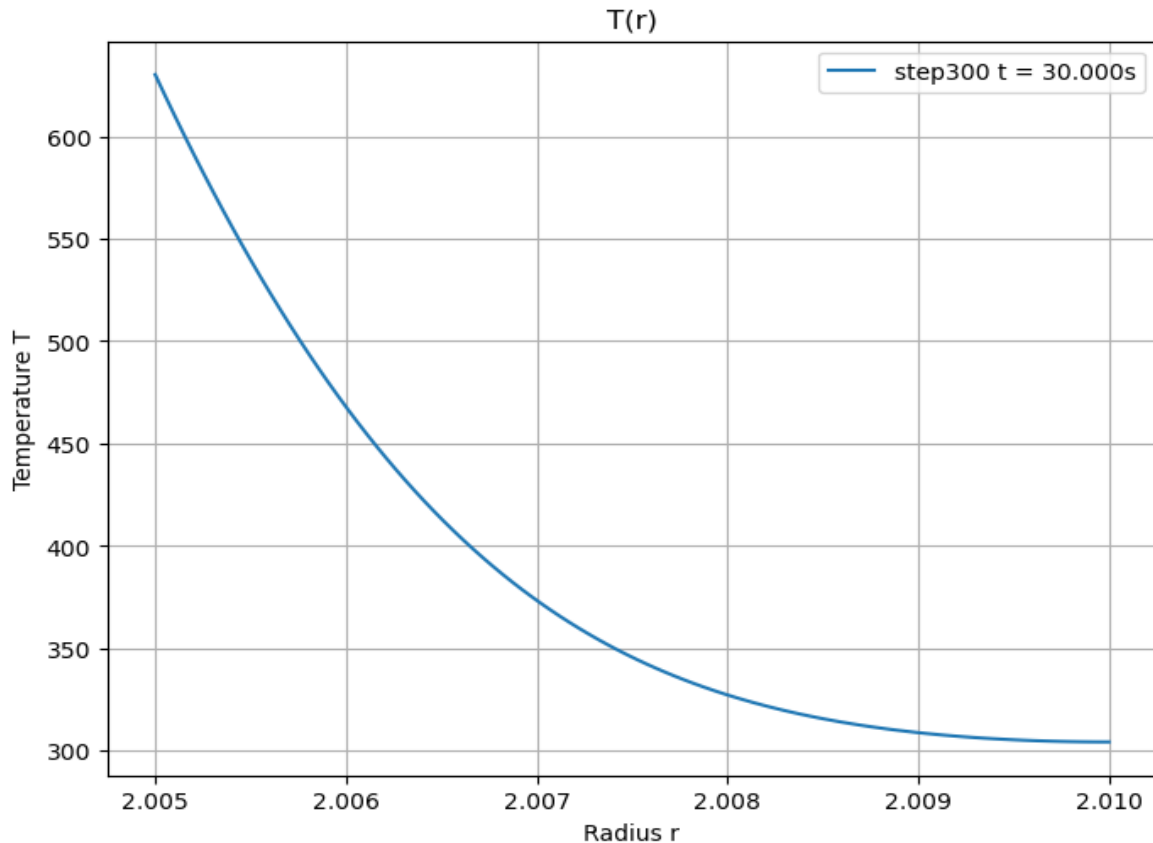


fig 11 : Plot of Temperature v/s time of the outer wall of the annular section (from Ansys APDL)

## 4.2 Python code

From the python code for solving Temperature using Implicit scheme, Finite Volume Method and TDMA (Tri-Diagonal Matrix Algorithm), the following results were obtained :

Temperature variation of the annular section with respect to radius after 30 seconds :



*fig 12 : Plot of Temperature v/s radius of the annular section (from Python code)*

- The temperature is varying gradually from inner wall (at radius = 2.005 m) to outer wall (at radius = 2.01 m).
- The highest temperature as can be seen from the plot above is at the inner surface of the annular section where flux of 20,000 W/m<sup>2</sup> is being applied.
- The lowest temperature is at the outer surface of the annular section where there is no flux applied (0.0 W/m<sup>2</sup>).
- The value of the highest temperature (temperature at the inner wall) is 630.0868080188566 K and the lowest temperature (temperature at the outer wall) is 304.3718519494044 K.

Plot of Temperature of the first cell (inner cell) varying with time:

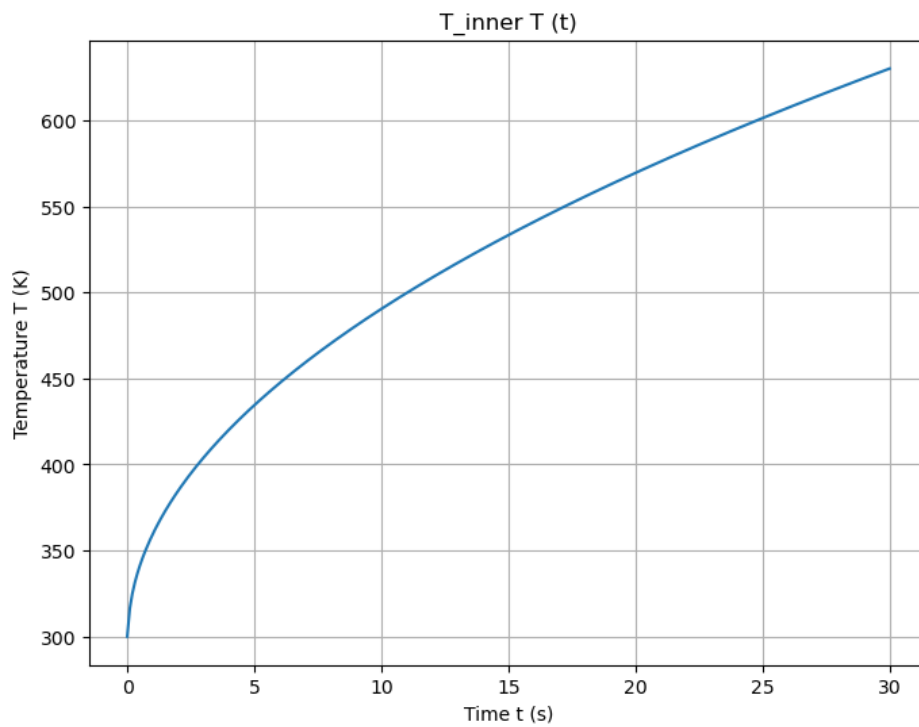


fig 13 : Plot of Temperature v/s time of the inner wall of the annular section (from Python code)

Plot of Temperature of the last cell (outer cell) varying with time:

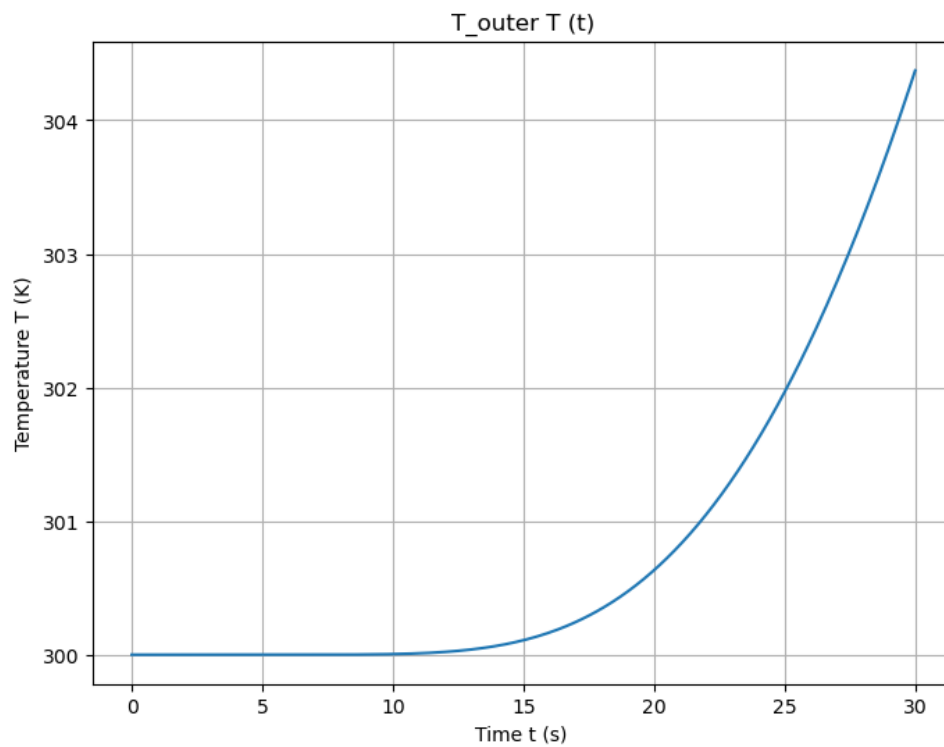
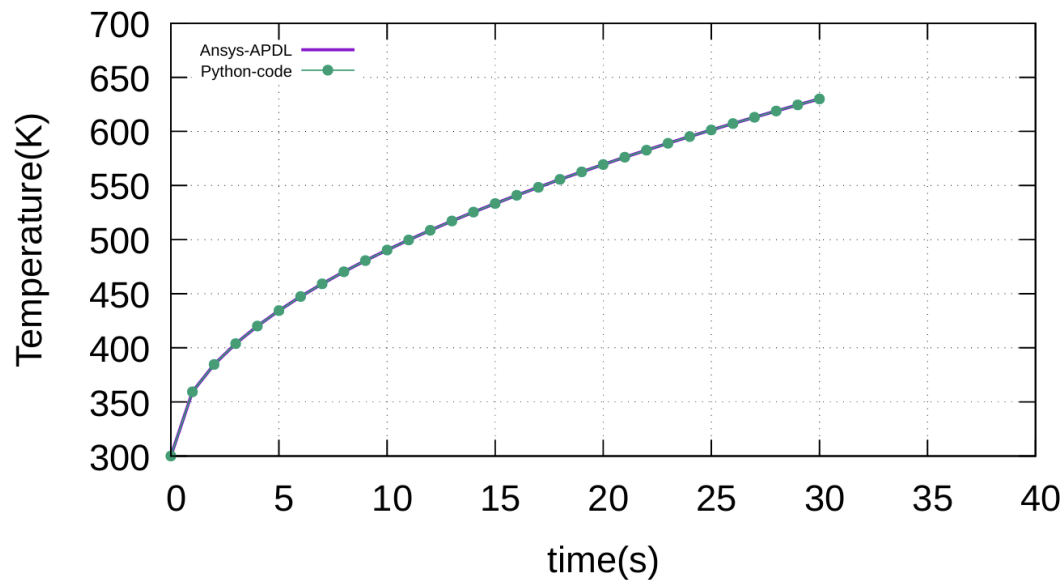


fig 14 : Plot of Temperature v/s time of the outer wall of the annular section (from Python code)

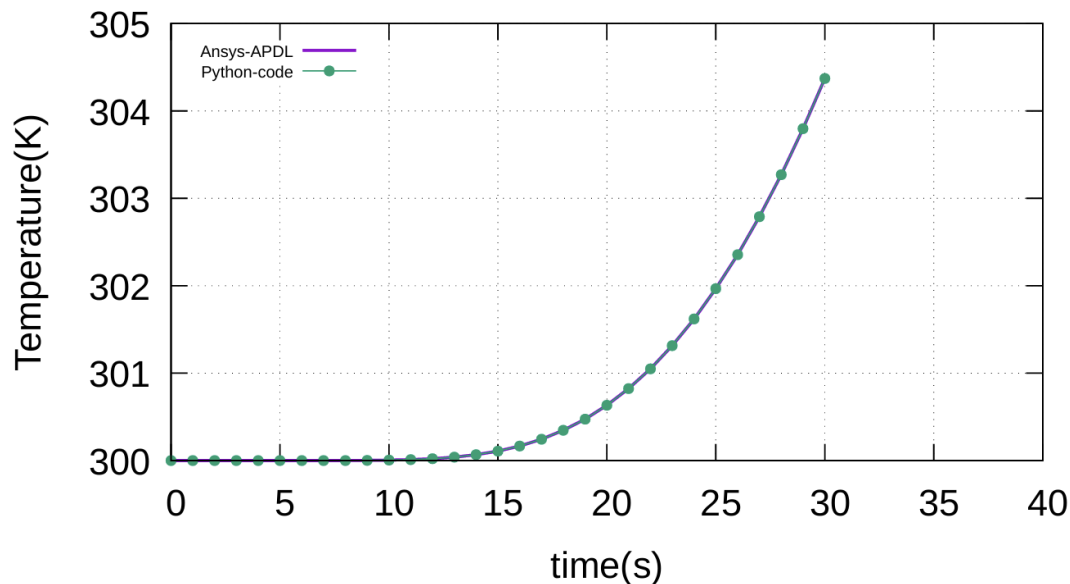
### 4.3 Verification

Overlapping the plots of Temperature of the first cell (inner cell) with respect to time taken from Ansys APDL and Python code:



*fig 15 : Overlapped plot of Temperature v/s time of the inner wall of annular section taken from Ansys APDL and Python code*

Overlapping the plots of Temperature of the last cell (outer cell) with respect to time taken from Ansys APDL and Python code:



*fig 16 : Overlapped plot of Temperature v/s time of the outer wall of annular section taken from Ansys APDL and Python code*

The plots matched with each other indicating the accuracy of the Python code and the simulation of Ansys APDL.

## TECHNOLOGICAL ADVANCEMENTS

Technological advancements in the Finite Volume Method (FVM) have greatly expanded its efficiency, accuracy, and scope of application, making it a cornerstone of modern computational science and engineering. Initially confined to structured meshes and basic discretization, FVM has evolved through the development of unstructured and hybrid grid systems, enabling accurate simulation of highly irregular and complex geometries such as those in aerospace structures, automotive components, biomedical flows, and energy systems. Higher-order discretization schemes, advanced flux reconstruction techniques, and adaptive mesh refinement strategies have significantly improved the resolution of steep gradients, boundary-layer interactions, and shock phenomena while maintaining computational efficiency.

The integration of high-performance computing (HPC) has been another transformative step. Parallelization strategies, GPU acceleration, and domain decomposition techniques have allowed FVM to tackle extremely large three-dimensional, time-dependent problems that were previously intractable. These computational advances have made real-time analysis, design optimization, and Multiphysics coupling feasible for both research and industrial applications. Furthermore, coupling FVM with advanced turbulence closures, multiphase models, and reactive flow solvers has enhanced its predictive capability for nonlinear, transient, and chemically reacting systems.

In recent years, FVM has also benefited from hybridization with complementary numerical methods, such as finite element and spectral schemes, to address multi-physics challenges where different discretizations are optimal for different domains. The integration of machine learning and data-driven modelling into FVM workflows has created opportunities for faster convergence, reduced computational cost, improved uncertainty quantification, and the development of surrogate models for complex design spaces. These innovations, along with the growing availability of open-source FVM platforms and visualization tools, have further democratized its use, encouraging adoption across academia, research, and industry.

Collectively, these advancements have elevated the Finite Volume Method from a classical discretization technique into a highly adaptive, scalable, and future-oriented computational framework, enabling accurate prediction, optimization, and innovation in a wide range of engineering and scientific fields.

## CONCLUSION/ SUMMARY

The numerical investigation of transient radial heat conduction in the annular wall of a nozzle demonstrated the successful application of modern computational approaches for thermal analysis. The study was carried out using two complementary methods: a Python-based solver employing the finite volume method (FVM) with an implicit scheme, and a high-fidelity Ansys APDL simulation. Both approaches captured the transient temperature distribution across the wall thickness, highlighting the strong thermal gradient generated by the constant heat flux at the inner wall and the near-steady temperature at the outer surface.

The Python solver, built upon FVM discretization and implemented with the Tri-Diagonal Matrix Algorithm (TDMA), proved to be both efficient and stable. The implicit formulation provided unconditional stability, enabling accurate predictions over the entire simulation period. In addition, the modular code structure allowed flexibility in defining input parameters such as geometry, material properties, and boundary conditions. This adaptability supports its application in rapid parametric studies, sensitivity analysis, and thermal design optimization.

The Ansys APDL simulations, performed using Quad 4-node 55 thermal elements and refined meshing, provided benchmark-quality results under identical physical conditions. The close agreement between Python and Ansys outputs—with maximum inner wall temperature  $\approx 630$  K and minimum outer wall temperature  $\approx 304$  K—validated the accuracy and reliability of the developed Python model. This cross-validation highlights the potential of open-source computational approaches as practical, cost-effective alternatives to commercial solvers while still maintaining engineering accuracy.

Overall, the study illustrates the technological advancements achieved in computational heat transfer analysis. The integration of Python-based solvers with commercial simulation tools creates a dual-framework that balances flexibility with industrial reliability. While Ansys APDL ensures high-fidelity modelling and visualization, the Python solver provides accessibility, adaptability, and reduced computational cost. This combination enhances confidence in engineering predictions and offers a pathway for extending such methods to more complex systems such as turbine blades, heat exchangers, combustion chambers, and other thermal management applications in aerospace and mechanical engineering.



## REFERENCES

- Introduction to Rocket Science and Engineering – Travis S Taylor
- Fundamentals of Compressible flow, with Aircraft and Rocket Propulsion – S M Yahya
- Rocket Propulsion Elements – George P Sutton, Oscar Biblarz
- Python for Scientists – John M Stewart
- Introduction to Computational Fluid Dynamics – Atul Sharma
- Computational Fluid Dynamics: Principles and Applications – J Blazek
- Fundamentals of Gas Dynamics – Robert D Zucker, Oscar Biblarz